CompressGrad: Quantifying the Compression–Reasoning Trade-off and Recovering Accuracy via TextGrad-Guided Repair of LLMLingua-Compressed Prompts

Krrish Singla *PES1UG24AM141*
Kishan K Shenoy *PES1UG24AM138*
Leeladhar G Sairaju *PES1UG24AM148*
Mohammad Bilal A *PES1UG24AM162*

Mini Project, Prompt Engineering and Optimization for Generative AI (UE24AM343AB5)
B.Tech Computer Science & Engineering (AI & ML), Semester 5, PES University, Bengaluru, India

***Abstract*—Prompt compression methods such as LLMLingua reduce the token cost of long-context LLM pipelines, but aggressive compression degrades answer quality. We measure this trade-off and test whether TextGrad-style textual feedback can recover part of the lost accuracy. On 100 HotpotQA questions, compressing the context by 2×, 5×, 10× and 20× with LLMLingua-2 lowers F1 from 0.731 (uncompressed, 1,452 prompt tokens on average) to 0.670, 0.483, 0.407 and 0.330. We then add a repair loop in which a judge model, which sees only the compressed context, the question and a draft answer (never the gold answer), writes textual feedback that TextGrad turns into a revised answer. A first version of the loop did not help and sometimes replaced answers with "unknown"; a second version, which forbids such answers and keeps drafts the judge approves, was evaluated on three sets of 30 questions. Pooled over 90 questions it raises F1 by +0.105 at 5× (about 41% of the lost accuracy recovered) and +0.114 at 10× (about 37%), with 95% bootstrap ranges that exclude zero. At 20× (60 questions) the gain is +0.027 and cannot be told apart from zero, which places the point where repair stops helping between 10× and 20×. These are pilot results from one dataset and one model pair, and the repair step costs extra model calls that we did not measure.**

***Index Terms*—prompt compression, LLMLingua, TextGrad, LLM-as-judge, question answering, prompt optimization**

I. Introduction

Large language models are increasingly used with long inputs such as documents, retrieved passages and enterprise records, and both API cost and latency grow with the number of prompt tokens. Prompt compression methods such as LLMLingua [1], LongLLMLingua [2] and LLMLingua-2 [3] shorten a prompt by removing tokens that a small model judges to carry little information, and they report large savings with limited loss on their benchmarks.

The accuracy side of this trade-off is less well mapped. Compression removes words the downstream model may need, and published results mostly describe compression on its own. Two questions remain open: at what compression ratio does answer quality fall off, and can part of the lost accuracy be recovered afterwards without sending the full prompt again?

TextGrad [4] offers one way to try. An LLM critiques a variable (here, a draft answer) in natural language, and this critique, treated as a "textual gradient," is used to revise the variable. The survey of papers and public repositories that we carried out for Review 1 found no work that combines TextGrad with LLMLingua-style compression, although we cannot rule out that such work exists.

We study three research questions. **RQ1:** How much accuracy does LLMLingua-2 compression cost at 2×, 5×, 10× and 20×? **RQ2:** Can TextGrad-guided repair, with a judge that never sees the gold answer, recover part of the lost accuracy? **RQ3:** At which compression ratio does repair stop helping?

This report makes four contributions:

- We measure the compression–accuracy curve of LLMLingua-2 on 100 HotpotQA questions at 2×, 5×, 10× and 20×.
- We build a compression-then-repair pipeline in which a judge model that sees only the compressed context, and never the gold answer, guides TextGrad to revise a draft answer.
- Through error analysis we explain why a first version of repair failed (the judge pushed answers towards "unknown" and edited answers that were already correct) and we fix it in a second version.
- We evaluate on question sets that were not used to design the method, with bootstrap ranges, and report where repair helps (5× and 10×) and where it stops helping (20×).

II. Related Work

A. Prompt Compression

LLMLingua [1] uses a small language model to identify and drop unimportant tokens. The compressed prompt is often hard for people to read but remains usable by the target LLM, and the paper reports compression of up to 20× with small loss on several benchmarks. LongLLMLingua [2] makes compression question-aware and targets the "lost in the middle" position bias [9] by reordering documents; it reports about 21% better performance with about 4× fewer tokens on NaturalQuestions and up to 94% cost reduction on LooGLE. LLMLingua-2 [3] treats compression as token classification, training a small encoder on data distilled from GPT-4, and is reported to be several times faster than LLMLingua. We use its XLM-RoBERTa-large model trained on MeetingBank. Selective Context [5] prunes tokens by self-information and is, like LLMLingua, not aware of the question. All of these works report the effect of compression itself. None of them tries to repair the answer afterwards.

B. Textual Gradients

TextGrad [4] treats natural-language feedback from an LLM as a gradient and passes it backwards through a computation graph, with an interface modelled on PyTorch. It has been used to optimize prompts, code and other text variables, and the authors report, for example, that it improves zero-shot GPT-4o accuracy on Google-Proof Question Answering from 51% to 55%.

In TextGrad a *Variable* holds a piece of text together with a short description of its role. A *TextLoss* wraps an evaluation instruction: given a variable, an LLM writes a critique of it. Calling *backward()* on the loss asks an LLM to turn that critique into feedback for each variable upstream, which is stored as the variable's gradient. The Textual Gradient Descent (*TGD*) optimizer then prompts an LLM with the variable, its gradient and optional constraints, and reads the improved value from between answer tags. In our pipeline the only variable is the draft answer, so the graph is short: draft, loss, feedback, revised draft.

C. LLM-as-Judge

Zheng et al. [6] show that strong LLM judges often agree with human raters but have known biases, namely position, verbosity and self-enhancement bias. In our system the judge produces feedback rather than the final score. Final scoring uses exact match and F1 against the gold answers, which removes judge bias from the evaluation, although not from the feedback itself.

D. Iterative Refinement with LLM Feedback

Self-Refine [10] lets a model critique and revise its own output in a loop without extra training, and Reflexion [11] stores verbal reflections on earlier failures to improve later attempts. TextGrad generalizes this idea by passing feedback through a graph of several variables. Our repair loop is a small instance of this family, in which feedback from a judge model is used to revise a draft answer. What is different here is the setting: the draft comes from a compressed prompt, and the judge sees only the compressed text.

E. Positioning

Table I summarizes how this work relates to the methods above, based on the survey we did for Review 1. The check marks show what each line of work does.

TABLE I
POSITIONING OF THIS WORK

Work	Compresses prompts	Repairs with text feedback	Both
LLMLingua, LongLLMLingua, LLMLingua-2	✓	–	–
TextGrad (general)	–	✓	–
TextGrad for SQL (course problem)	–	✓	–
CompressGrad (this work)	✓	✓	✓

III. Method

A. Pipeline Overview

Given a context C and a question q, the compressor returns a shorter context C_r with roughly $|C|/r$ tokens. The answer model produces a draft answer a_0 from (C_r, q). The repair step then produces a revised answer a^* from (C_r, q, a_0). We compare three conditions per compression level: the answer from the full context, the draft a_0, and the repaired a^*. Only the context is compressed; the instruction and the question are left intact. Fig. 1 shows the pipeline. A design principle throughout is that nothing from the gold answers or the original context is available at repair time, so the repair step could be applied to new questions in the same way.

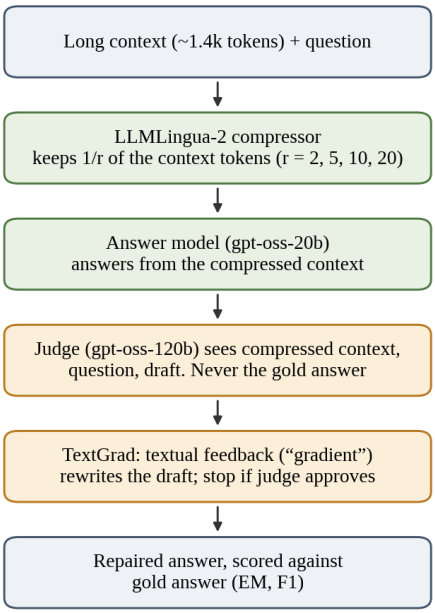


Fig. 1. The compression-then-repair pipeline. The judge never receives the gold answer.

B. Compression

We use LLMLingua-2 through the *llmlingua* Python package with the *llmlingua-2-xlm-roberta-large-meetingbank* model. LLMLingua-2 labels each token of the input as keep or drop with a bidirectional encoder that was trained on text compressed by GPT-4 from the MeetingBank corpus, and it keeps the tokens with the highest keep probability until the requested budget is reached, so the order of the original words is preserved. For a ratio r the target keep rate is $1/r$; newlines are forced to be kept and consecutive duplicates are dropped. Because the compressor and the answer model use different tokenizers, we report the average prompt tokens counted by the answer model's API. These counts include the fixed instruction and the question, so the real token reduction is smaller than r (Table III).

As an illustration, a short test passage was "*The city council met on Tuesday to discuss the new transport budget. After three hours of debate, the council voted to increase bus funding by*

twelve percent, while delaying the planned metro extension until next year." (42 tokens). At a keep rate of 0.5 LLMLingua-2 returned "*city council Tuesday transport budget three hours debate voted increase bus funding twelve percent metro extension next year*" (18 tokens). The key facts survive, but function words and some details are gone, and this loss is what the repair step has to compensate for.

C. TextGrad Repair Loop

The draft answer is a TextGrad *Variable* with gradients enabled. A *TextLoss* is built from a fixed evaluation instruction that contains the question and the compressed context. It asks the judge whether the draft is supported by the context, answers exactly what the question asks, and is short. Calling *backward()* makes the judge model write feedback on the variable (the textual gradient), and the *TGD* optimizer step rewrites the answer using that feedback. The judge and optimizer share one model and never see the gold answer or the original context. We evaluate three versions.

The gradient step uses TextGrad's own fixed system prompt. It tells the model that it is part of an optimization system, that it should give feedback on the variable without writing a new version of it, and that a variable which is already working well should receive no feedback. The rewriting is left to the optimizer step.

v1 uses the evaluation instruction above and one repair round.

v2 adds three rules to the instruction and optimizer: (i) never answer "unknown" or "not stated", always give the best guess the context supports; (ii) if the draft is correct, reply "The draft answer is correct.", which stops the loop and keeps the draft; (iii) keep the answer in the same short form, without descriptions or brackets. A guard also keeps the draft if the final answer still contains an abstention phrase. The guard never fired in any of our runs, so the instructions alone removed these answers.

v3 adds to v2 the rules that the judge may only change the draft when the context clearly supports another answer and that a new answer must use words that appear in the context, and it runs two rounds instead of one.

Algorithm 1 shows the repair of one answer. In v1 the loop has no early stop and no guard, and in v3 the number of rounds R is two.

```
Algorithm 1: Repair of one answer (v2)
Input: question q, compressed context Cr,
       draft a0, judge/optimizer model J
 1: a <- Variable(a0, requires_grad=True)
 2: L <- TextLoss(instruction(q, Cr), J)
 3: for round = 1 .. R:
 4:   fb <- L(a)          # judge reads a
 5:   if fb says draft is correct: break
 6:   fb.backward()       # textual gradient
 7:   TGD.step()          # J rewrites a
 8: if a contains an abstention phrase:
 9:   a <- a0             # guard
10: return a
```

D. Metrics

We use exact match (EM) and token-level F1. Both first normalize the text by lowercasing it, removing punctuation and the articles *a*, *an* and *the*, and collapsing spaces. EM is 1 when the normalized prediction equals the normalized gold answer and 0 otherwise. For F1 the prediction and the gold answer are treated as bags of words: precision is the number of shared words divided by the number of predicted words, recall is the same count divided by the number of gold words, and F1 is their harmonic mean, which gives partial credit for answers that overlap with the gold answer. Both scores are averaged over questions. The share of lost accuracy that repair wins back is

$$Recovery = (F1_{repaired} - F1_{comp.}) / (F1_{full} - F1_{comp.}) \quad (1)$$

computed on the same questions in each case. To judge whether an average change differs from zero, we resample the per-question differences, as Algorithm 2 shows.

```
Algorithm 2: Bootstrap range for the F1 change
Input: differences d1..dn (repaired F1 minus
       draft F1) for n questions
 1: for b = 1 .. 5000:
 2:   draw n values from d with replacement
 3:   m[b] <- mean of the drawn values
 4: sort m
 5: return m[125], m[4875]   # 95% range
```

E. Middleware and Demo

To package the method we provide a function, *compress_and_repair(context, question, ratio)*, which compresses the context with LLMLingua-2, answers from the compressed context, applies repair v2 and returns the compressed context, the token counts, the draft answer and the final answer. A demo script calls it on a question from the sample or on a text file supplied by the user. It prints the answers from the full context, from the compressed context and from the compressed context with repair next to each other, together with their F1 scores when a gold answer is known. Both files are listed in Appendix B.

IV. Experimental Setup

A. Data

We use the distractor setting of HotpotQA [7], a multi-hop question answering dataset in which each question comes with ten paragraphs, two of which contain the supporting facts. We sampled 100 questions at random (seed 42) from the 7,405-question validation split. With the instruction and question, an uncompressed prompt has about 1,450 tokens on average. As an example, the first question in our sample asks in what year the university where Sergei Aleksandrovich Tokarev was a professor was founded (gold answer: 1755). It needs two steps: finding the university and then finding its founding year. Its context has 406 words in ten paragraphs, and the uncompressed baseline answers it correctly.

B. Models and Infrastructure

The answer model is gpt-oss-20b and the judge and optimizer is gpt-oss-120b [8], both served through the free tier of the Groq API with temperature 0 and low reasoning effort. Every model call is cached, so a rerun gives the same results. The free tier limits the tokens per day, which is why the repair experiments use sets of 30 questions.

C. Evaluation Protocol

The compression curve uses all 100 questions. Repair is evaluated on three disjoint sets of 30 questions each. Questions 1–30 are the development set: we used them to find the failure modes of v1 and to design v2. Questions 31–60 are held out for v2. Questions 61–90 were not used until after v3 had been designed, and we ran both v2 and v3 on them. Repair was run at 5× and 10× on all three sets and at 20× on the first two. Fig. 7 shows which version was run on which questions. We estimate the uncertainty of the average F1 change with a paired bootstrap (5,000 resamples of the per-question differences) and report the 2.5th to 97.5th percentile as a 95% range.

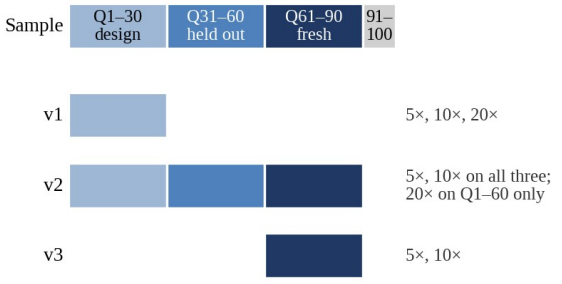


Fig. 7. Question sets and the repair versions evaluated on each. Questions 91–100 were not used for repair.

TABLE II
Experimental settings

Item	Setting
Dataset	HotpotQA distractor setting, validation split
Sample	100 questions, random seed 42
Compressor	LLMLingua-2, XLM-RoBERTa-large (MeetingBank)
Levels	2×, 5×, 10×, 20× (keep rate 1/r)
Answer model	gpt-oss-20b via Groq
Judge / optimizer	gpt-oss-120b via Groq
Decoding	Temperature 0, low reasoning effort
Repair	TextGrad TextLoss and TGD; 1 round (v1, v2), 2 rounds (v3)
Metrics	EM, F1; 5,000-resample paired bootstrap

D. Engineering Challenges

Several practical problems shaped the final design. **Model retirement.** Our first choice of model on Groq, llama-3.3-70b-versatile, was retired on 16 August 2026 and requests to it failed with an HTTP 404 error, so we moved to gpt-oss-20b and gpt-oss-120b. **Daily token limits.** The free tier allows 200,000 tokens per day for each model. An uncompressed run over 100 questions needs about 145,000 prompt tokens (100 × 1,452), which is about 73% of one model's daily allowance before any output tokens are counted, while the same run at 5× needs about 38,000. Our compressed-only evaluation hit the limit part-way through the 2× level, so we added detection of the daily limit, a clean stop and resume from the saved responses. **No GPU.** The compressor failed with a CUDA memory error on a machine without a usable GPU, so we run it on the CPU, and a free cloud GPU is an alternative for this step. **Library integration.** TextGrad's built-in engine names did not work with our provider string, so we wrote a small engine class that routes TextGrad's calls through our cached caller. **Reasoning models.** The gpt-oss models spend tokens on hidden reasoning, so we set a low reasoning effort and a token limit that leaves room for the visible answer.

E. Implementation

The pipeline is a set of Python scripts: one prepares the 100-question sample, one compresses the contexts at each level, one scores the baseline and the compressed-only answers, one runs the repair loop (versions v1 to v3 and the question sets are selected by command-line options), and two produce the statistics and the figures. Every model response is stored in a local cache keyed by the model and the prompt, and the repair results are written one question at a time, so an interrupted run resumes where it stopped.

For each question and level the pipeline makes one answer call (shared with the compressed-only evaluation through the cache) and one judge call to evaluate the draft. If the judge does not approve the draft, one further call produces the gradient and one more rewrites the answer, for each round. In v1 every draft goes through these calls, and in v2 and v3 drafts that the judge approves stop early. The scripts handle rate limits by waiting and retrying and stop cleanly, with progress saved, when the

daily token limit of the free tier is reached. The experiments were run on a Windows laptop in October 2026 with llmlingua 0.2.2, textgrad 0.1.8, litellm 1.106.2 and datasets 5.1.0.

V. Results

A. Compression Alone

Table III and Fig. 2 show the compression–accuracy curve. Going from the full context to 2× costs 0.061 F1 while removing about 47% of the prompt tokens. Beyond that, F1 falls faster: 0.483 at 5× (74% fewer tokens), 0.407 at 10× (83%) and 0.330 at 20× (87%). Exact match follows the same pattern. So 2× is close to free, while 5× to 20× leave a large gap that repair could try to close. Measured per 1,000 prompt tokens, F1 rises from 0.50 for the full prompt to 0.88, 1.26, 1.62 and 1.81 at 2×, 5×, 10× and 20×, so compression always makes the prompt more token-efficient, and accuracy is what pays for it.

TABLE III
Compression only (HotpotQA, n = 100)

Level	Prompt tokens	Token reduction	EM	F1	F1 kept
None	1,452	–	0.530	0.731	100%
2×	765	47%	0.490	0.670	92%
5×	383	74%	0.340	0.483	66%
10×	251	83%	0.320	0.407	56%
20×	182	87%	0.230	0.330	45%

Prompt tokens are averages counted by the answer model and include the instruction and question. “F1 kept” is F1 divided by the uncompressed F1.

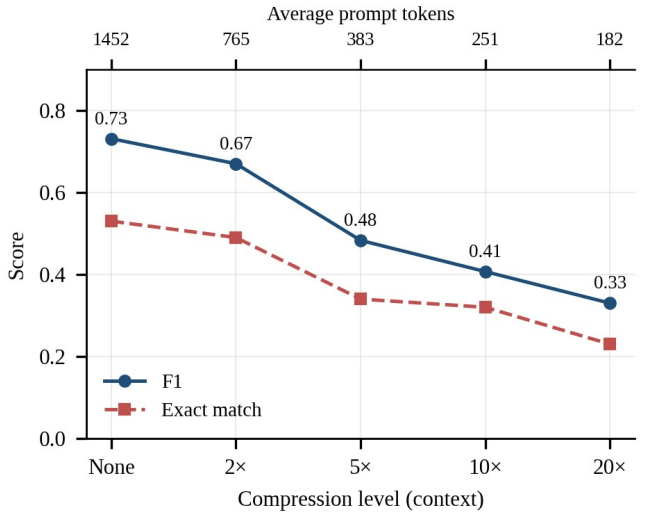


Fig. 2. Accuracy against compression level on 100 questions. The top axis shows the average prompt tokens.

B. Repair v1 Did Not Help

On questions 1–30, v1 changed F1 by +0.033, +0.027 and −0.050 at 5×, 10× and 20×. The number of questions it helped and hurt was 5 and 5, 6 and 6, and 7 and 8. The 95% ranges all include zero, so there was no evidence of a gain at any level.

C. Repair v2

Table IV gives the v2 results for each question set and Table V pools them. At 5× v2 recovered 31.9%, 46.0% and 41.4% of the lost F1 on the three sets, and at 10× it recovered 42.2%, 33.1% and 33.9%. On the two sets not used for design, the F1 gain was +0.139 and +0.108 at 5× and +0.129 and +0.078 at 10×. At 20× the result was mixed (16.1% on the first set, −1.7% on the second).

TABLE IV
Repair v2 by question set

Level	Qs	F1 comp.	F1 rep.	EM comp.	EM rep.	Recovered
5×	1–30	0.543	0.610	0.400	0.467	31.9%
5×	31–60	0.492	0.631	0.400	0.533	46.0%
5×	61–90	0.390	0.498	0.233	0.367	41.4%
10×	1–30	0.431	0.567	0.367	0.433	42.2%
10×	31–60	0.406	0.535	0.333	0.467	33.1%
10×	61–90	0.424	0.502	0.300	0.400	33.9%
20×	1–30	0.379	0.439	0.233	0.333	16.1%
20×	31–60	0.406	0.399	0.333	0.367	−1.7%

Qs = question numbers (30 per set). “Recovered” is computed on F1 with Eq. (1). Questions 1–30 were used to design v2. 20× was not run on questions 61–90.

Pooled over all questions (Table V, Fig. 3), v2 raises F1 by +0.105 at 5× and +0.114 at 10×, and the 95% ranges exclude zero. Exact match rises from 0.344 to 0.456 at 5× and from 0.333 to 0.433 at 10×. At 20× the pooled gain is +0.027 and the range includes zero.

TABLE V
Repair v2 pooled over question sets

Level	n	Helped / hurt / same	ΔF1 [95% range]	Recovered
5×	90	16 / 5 / 69	+0.105 [+0.026, +0.187]	≈41%
10×	90	18 / 5 / 67	+0.114 [+0.032, +0.199]	≈37%
20×	60	7 / 3 / 50	+0.027 [−0.047, +0.102]	≈7%

ΔF1 is the average per-question F1 change from the compressed-only draft to the repaired answer.

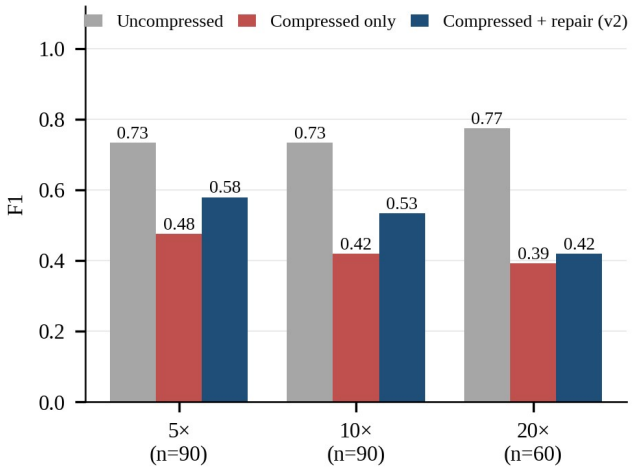


Fig. 3. Uncompressed, compressed-only and repaired (v2) F1 on the pooled question sets. Each level uses its own set of questions, so the uncompressed bars differ.

D. Comparing the Versions

Table VI compares the three versions. On questions 61–90, v3 scored higher than v2 at 5× (0.523 against 0.498) and lower at 10× (0.457 against 0.502). With 30 questions neither

difference is clear, and v3 needs up to two rounds of model calls. We therefore treat v2 as the final method and v3 as an ablation.

TABLE VI
AVERAGE F1 CHANGE FROM REPAIR, BY VERSION

Version	Questions	5×	10×	20×
v1	1–30	+0.033	+0.027	−0.050
v2	1–30	+0.067	+0.136	+0.060
v2	61–90	+0.108	+0.078	–
v3	61–90	+0.133	+0.033	–

E. Error Analysis

We found three failure modes.

Abstention. With v1, the judge turned drafts into answers such as “Not stated”, “Cannot be determined” and “Insufficient data”. None of the drafts were abstentions, but 2, 3 and 8 of the repaired answers at 5×, 10× and 20× were. The stronger the compression, the less the judge can verify, and the more often it gives up. The v2 rules removed this completely.

Editing correct answers. The loop is built to improve its variable, so it also edits answers that are already right: “Gatwick Airport” became “London Gatwick”, “dogs” became “dog breeds” and “Kevin Spacey” became “Kevin Spacey (actor)”. These examples come from v1. With the “draft is correct” rule in v2, repair left the F1 score unchanged for about three quarters of the questions (69 of 90 at 5×, 67 of 90 at 10×).

Entity swaps. Some errors remain in v2 and v3, for example “Rudolf Höss” became “Kurt Gerstein”, “Brady Seals” became “Nick Carter” and “Cold War” became “World War I”. When compression removes the sentence that links two entities, the judge can pick a wrong but plausible entity from the remaining text. The v3 rule to use only words from the context does not prevent this, because the wrong entity is in the context.

Repairs also work as intended: the draft “Norwegian” became “Punjabi/Pashtun” and the draft “Logar” became “Pul-i-Alam”, which matched the gold answers.

TABLE VII
EXAMPLES OF REPAIRED ANSWERS

Case	Gold	Draft	Repaired
v2, 5×, helped	Punjabi/Pashtun	Norwegian	Punjabi/Pashtun
v2, 5×, helped	Puli Alam	Logar	Pul-i-Alam
v1, 10×, gave up	Greyia	Greyia	cannot be determined
v1, 20×, over-edit	dogs	Dogs	dog breeds
v1, 20×, over-edit	Kevin Spacey	Kevin Spacey	Kevin Spacey (actor)
v2, 5×, swap	the Cold War (1947–91)	Cold War	World War I
v3, 5×, swap	Rudolf Höss	Rudolf Höss	Kurt Gerstein
v2, 20×, swap	Brady Seals	Brady Seals	Nick Carter
v2, 10×, helped	dinosaur	Red and white “S” logo.	dinosaur
v2, 5×, hurt	Blood and soil	Blood and Soil (Blut und Boden)	Blut und Boden
v2, 10×, swap	Georgetown University	University of Michigan	Missouri
v2, 10×, swap	Chaerophyllum	Chaerophyllum	Cryptanthus
v2, 20×, swap	Dirk Werner Nowitzki	Dirk Nowitzki	Lukas Kanter
v3, 10×, swap	evangelical Christian	Christian (Protestant)	Wesleyan

Examples were taken from the result files; they are illustrative, not a random sample.

F. Where Repair Stops Helping

In our setup repair helps at 5× and 10× but not reliably at 20×, so the breakdown point lies between 10× and 20×. One explanation is that at 20× only about 80 context tokens remain, which gives a judge that sees only the compressed text little to check the draft against. We did not test this explanation directly. Fig. 4 puts the results on a token-versus-accuracy plot: at 5×, repair moves F1 from 0.475 to 0.580 with the same answer-call prompt size of 383 tokens. This plot does not include the tokens used by the judge and optimizer calls.

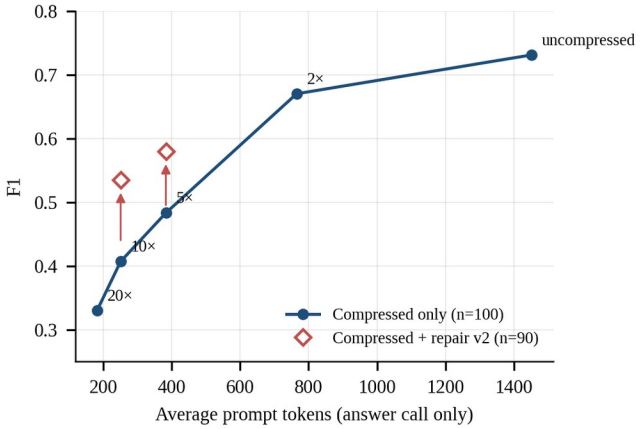


Fig. 4. Token cost against accuracy. Repaired points use the pooled 90 questions, and the arrows start from the compressed-only F1 on those same questions. Repair-call tokens are not counted.

G. Consistency Across Question Sets

Fig. 5 shows the share of lost F1 that v2 recovered on each question set. At 5× and 10× the recovery stays between 31.9%

and 46.0% on all three sets, including the two that were not used to design the method. At 20× it is 16.1% on the first set and −1.7% on the second, which is the reason we do not claim a gain there. Because every set has only 30 questions, a single question can change a set's F1 by about 0.03, so differences of a few points between sets are within what we would expect from chance.

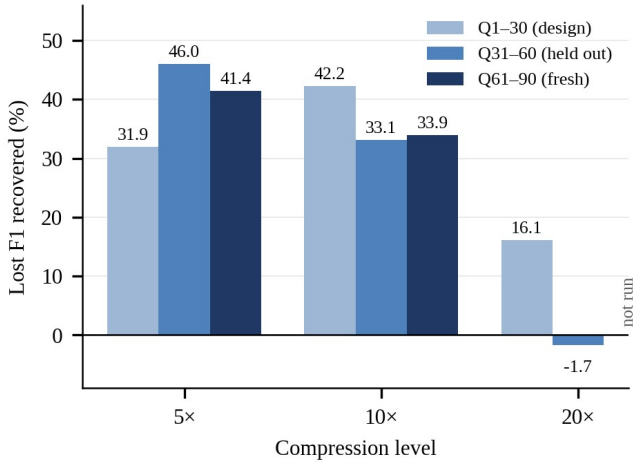


Fig. 5. Share of the lost F1 recovered by repair v2 on each set of 30 questions. The 20× level was not run on questions 61–90.

H. Outcome Counts per Run

Fig. 6 compares v1 and v2 on the same 30 questions by counting the questions where repair raised the F1, lowered it, or left it unchanged. v1 hurt as many questions as it helped (5 and 5, 6 and 6) or more (7 helped and 8 hurt at 20×). v2 hurt one question or none at each level and left about four in five questions unchanged. Table VIII lists every run, with the 95% range where we computed it.

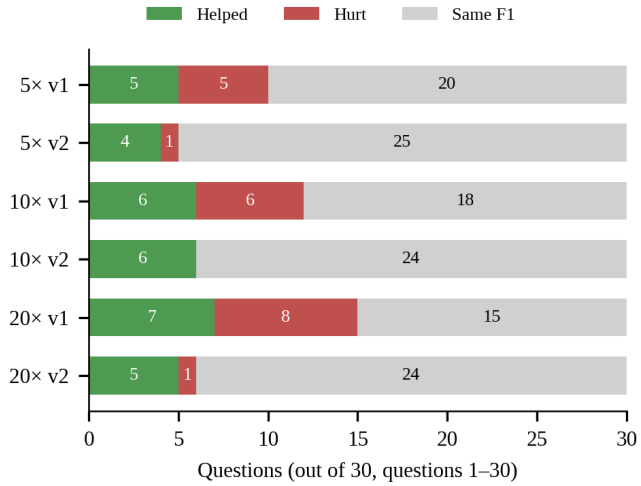


Fig. 6. Questions helped, hurt and unchanged by repair on questions 1–30, for v1 and v2.

TABLE VIII
ALL REPAIR RUNS

Version	Qs	Level	Helped / hurt / same	ΔF1 [95% range]
v1	1–30	5×	5 / 5 / 20	+0.033 [−0.119, +0.187]
v1	1–30	10×	6 / 6 / 18	+0.027 [−0.143, +0.187]
v1	1–30	20×	7 / 8 / 15	−0.050 [−0.229, +0.131]
v2	1–30	5×	4 / 1 / 25	+0.067 [−0.024, +0.180]
v2	1–30	10×	6 / 0 / 24	+0.136 [+0.036, +0.249]
v2	1–30	20×	5 / 1 / 24	+0.060 [−0.029, +0.156]
v2	31–60	5×	6 / 1 / 23	+0.139 [+0.000, +0.300]
v2	31–60	10×	6 / 2 / 22	+0.129 [−0.024, +0.284]
v2	31–60	20×	2 / 2 / 26	−0.007 [−0.120, +0.100]
v2	61–90	5×	6 / 3 / 21	+0.108 [–]
v2	61–90	10×	6 / 3 / 21	+0.078 [–]
v3	61–90	5×	6 / 2 / 22	+0.133 [−0.013, +0.293]
v3	61–90	10×	4 / 3 / 23	+0.033 [−0.093, +0.157]

The 95% range was not computed for v2 on questions 61–90. The counts for that run follow from the pooled counts in Table V.

I. Cost Considerations

The comparisons above use the prompt tokens of the answer call. Repair adds model calls, and these carry text too. In TextGrad, the judge call that evaluates the draft and the gradient call that turns the critique into feedback both receive the evaluation instruction, which contains the compressed context. A draft that the judge does not approve therefore costs at least two calls that re-send the compressed context, plus a third call that rewrites the answer. At 5× the answer call has 383 prompt tokens against 1,452 for the full prompt, a saving of 1,069 tokens per question. How much of this saving repair keeps depends on how often the judge approves the draft after the first call and on how long its feedback is. We did not measure this for the results above. The repository contains a script, *cost_analysis.py*, that replays the stored calls and adds up the prompt tokens of the repair step for each level, and any claim about the cost of repair should rely on its output.

J. Summary of Results

- Compression by 2× keeps 92% of the F1 (0.670 against 0.731) with 47% fewer prompt tokens. At 5×, 10× and 20× F1 falls to 0.483, 0.407 and 0.330.
- Repair v1 did not help: it hurt about as many questions as it helped, and at higher compression it produced "unknown" answers.
- Repair v2 raised F1 by +0.105 at 5× and +0.114 at 10× over 90 questions, recovering about 41% and 37% of the lost F1, with 95% ranges that exclude zero.
- At 20× the gain was +0.027 over 60 questions and cannot be told apart from zero. The breakdown point of repair lies between 10× and 20×.
- v3, with two rounds and a stricter rule, did not reliably beat v2 on questions 61–90.

Sample size and scope. Each repair set has 30 questions, giving 90 questions at 5× and 10× and 60 at 20×. We used one dataset, one task (question answering) and one pair of models,

so the results may not carry over to summarization, which Review 1 also proposed, or to other models.

Reuse of questions. v2 was designed on questions 1–30, and the pooled numbers include them. The held-out sets alone show gains of +0.139 and +0.108 at 5× and +0.129 and +0.078 at 10×. v3 was designed after we had seen the results on questions 1–60.

Repair cost. The repair step is not free. Each draft that the judge does not approve needs up to three further model calls per round (evaluation, feedback and rewrite) with a larger model, and these prompts contain the compressed context. The total saving can therefore be much smaller than the 74–83% reduction in answer-call tokens. We did not measure the total tokens or the latency, although Review 1 listed cost and latency savings as expected outcomes.

Source of the gain. The judge is a larger model than the answer model and may use its own world knowledge, so part of the gain may come from a second opinion by a stronger model and not from recovering lost context. We did not run the control in which the larger model answers directly from the compressed context.

Other limits. We used LLMLingua-2 only, not the question-aware LongLLMLingua, and we scored with EM and F1, not with an LLM judge as Review 1 planned. F1 gives partial credit for overlapping words. The Groq free tier limited the number of questions, and hosted models can change over time.

Why repair helps. We did not test why repair helps at 5× and 10×. Two explanations are possible and our data cannot separate them: the judge may find information that is still present in the compressed text but was missed in the first answer, or it may bring in knowledge of its own, since it is a larger model. The control described in the future work would separate the two.

Data and responsible use. HotpotQA is a public dataset released under a Creative Commons licence, and its text comes from Wikipedia. We used the free tier of a commercial API within its limits. Our results are a pilot study and should not be read as a general measurement of LLMLingua-2 or TextGrad.

Data contamination. HotpotQA is public, and the gpt-oss models may have seen it during training. This affects all conditions, but it could make compressed-only answers look better than they would on unseen data, and it could help the judge guess an answer from memory.

Repeatability. Hosted models can change and decoding at temperature 0 is not guaranteed to be identical across runs. Our cache makes the reported numbers repeatable from the stored responses, but a fresh run with new model versions may differ.

Scaling plan. With 90 questions the 95% range of the average F1 change at 5× runs from +0.026 to +0.187 around +0.105, which is about 0.08 on each side. Such a range shrinks with the square root of the number of questions, so about 360 questions would be needed to halve it. At the speed of our runs the free tier would need many days for that, so a paid tier or a smaller judge model would help.

Practical guidance. The results suggest two operating points. For accuracy-critical use, 2× compression without repair keeps F1 at 0.670, which is about 92% of the uncompressed 0.731, while using 47% fewer prompt tokens. For cost-critical use, 5× compression with repair reaches an F1 of about 0.58, about 79% of the uncompressed F1 on the same questions, with 74% fewer tokens in the answer call, as long as the extra repair calls are acceptable. At 10× and beyond, compression alone loses a lot of accuracy and our repair design recovers only part of it.

VII. Comparison with the Review 1 Plan

Table IX compares the plan from Review 1 with what we did. The main differences are that we used a public question-answering dataset in place of enterprise documents, did not run the summarization task, scored with exact match and F1 in place of an LLM judge, and did not measure latency.

TABLE IX
Review 1 plan and what was done

Planned	Done	Status
Literature review, setup	Survey of compression, TextGrad and LLM-as-judge; environment	Done
Dataset and baseline	100 HotpotQA questions, not enterprise documents; uncompressed baseline	Changed
LongLLMLingua and LLMLingua-2	LLMLingua-2 only	Partly
Compression tiers, repair loop	2× to 20×; TextGrad repair v1 to v3	Done
QA and summarization	Question answering only	Partly
LLM-judge scoring, latency, cost	EM, F1, bootstrap ranges, answer-call tokens; no latency; replay script for repair cost	Partly
Middleware, demo, report	Pipeline function, demo script, this report	Done

VIII. Conclusion

Compressing the context of HotpotQA prompts by 2× costs little accuracy, but F1 falls from 0.731 to 0.483, 0.407 and 0.330 at 5×, 10× and 20×. A TextGrad repair loop guided by a judge that sees only the compressed context recovers about 41% of the lost F1 at 5× and about 37% at 10× once it is told to avoid “unknown” answers and to leave correct drafts alone, but it does not help reliably at 20×. The first version of the loop failed for reasons that error analysis made visible, which is a useful result in itself.

In terms of the research questions: for RQ1, compression costs 0.061 F1 at 2× and 0.401 at 20× (0.731 down to 0.330). For RQ2, repair v2 recovers about 41% of the lost F1 at 5× and about 37% at 10×. For RQ3, repair stops helping reliably between 10× and 20×.

We see five directions for future work. First, scale the experiments to the full question set, to all compression levels

and to more than one dataset, so that the ranges become narrower. Second, add the control in which the larger judge model answers directly from the compressed context, to separate the effect of repair from the effect of a stronger model. Third, measure the total tokens and the latency including all repair calls, to learn the real cost saving. Fourth, try summarization and the question-aware LongLLMLingua, as proposed in Review 1. Fifth, test a repair step that is allowed to restore a small number of tokens from the original context within a fixed budget, which could help in the entity-swap cases where the needed sentence was removed by compression.

IX. Team and Contributions

The project was carried out by a team of four students. In Review 1 each member presented one part of the work, as shown in Table X.

TABLE X
TEAM MEMBERS AND REVIEW 1 RESPONSIBILITIES

Member	SRN	Review 1 section
Krrish Singla	PES1UG24AM141	Problem, motivation and research gap
Kishan K Shenoy	PES1UG24AM138	Literature survey: prompt compression
Leeladhar G Sairaju	PES1UG24AM148	Literature survey: optimization, evaluation and the gap
Mohammad Bilal A	PES1UG24AM162	Objective, technical approach and expected outcome

Final implementation, experiments and report: [TEAM TO COMPLETE: who did the compression, repair loop, evaluation, figures and writing].

Code, data sample and results: [GITHUB URL].

Acknowledgment

The authors thank the faculty of the course Prompt Engineering and Optimization for Generative AI (UE24AM343AB5) at PES University for their guidance during the reviews.

Appendix A: Prompts

The answer prompt, with the (compressed) context and the question inserted, was:

> *Answer the question using the context below. Reply with ONLY the short answer (a few words), no explanation. Context: [context] Question: [question] Answer:*

The evaluation instruction given to the judge, with the question and the compressed context appended, was:

> *You are a strict judge of a question-answering system. You get a QUESTION, a CONTEXT and a DRAFT ANSWER. The CONTEXT is a heavily compressed version of the original text, so some words are missing but most key facts remain. Check the draft answer: Is it supported by the context? Does it answer exactly what the question asks? Is it short? Point out specific problems and say what the correct short answer should be, based on the context. Never say the answer is unknown, not stated or cannot be determined; always give the best guess the context supports. If the draft answer is already correct and complete, reply with exactly: The draft answer is correct. Keep the same form: just the name, date, number or place, with no descriptions, brackets or extra words.*

v3 adds: *Only suggest a different answer if the context clearly supports it for this exact question. Any new answer must use words that appear in the context.*

Appendix B: Repository and Reproduction

Table XI lists the files of the project repository, and the commands below reproduce the experiments. All commands are run from the repository folder with the environment from *requirements.txt* and the Groq key set in the *GROQ_API_KEY* variable.

TABLE XI
REPOSITORY FILES

File	Purpose
config.py	Models, sample size, compression levels
utils.py	Cached model calls, retries, scoring
prepare_data.py	Samples 100 HotpotQA questions
compress.py	LLMLingua-2 at 2× to 20×
baseline.py	Uncompressed baseline
eval_compressed.py	Compressed-only evaluation
repair.py	TextGrad repair (v1, v2, v3)
analyze_repair.py	Helped/hurt counts, bootstrap ranges
make_plots.py	Figures from the result files
cost_analysis.py	Prompt tokens used by repair (replay)
pipeline.py	compress_and_repair() middleware
demo.py	Side-by-side demo of the three answers

```
Commands
python prepare_data.py
python compress.py
python baseline.py
python eval_compressed.py
python repair.py v2            # Q1-30
python repair.py v2 heldout    # Q31-60
python repair.py v2 final ratios=5,10
python repair.py v3 final ratios=5,10
python analyze_repair.py v2 all
python make_plots.py
python cost_analysis.py        # offline
python demo.py --ratio 5       # demo
```

Appendix C: Notation

TABLE XII
SYMBOLS USED IN THE REPORT

Symbol	Meaning
C	Original context (about 1,450 prompt tokens)
q	Question
r	Compression ratio (2, 5, 10 or 20)
C_r	Compressed context with about $\|C\|/r$ tokens
a_0	Draft answer from (C_r, q)
a^*	Repaired answer
R	Number of repair rounds (1 for v1 and v2, 2 for v3)
ΔF1	F1 of a^* minus F1 of a_0, per question

References

[1] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, and L. Qiu, "LLMLingua: Compressing prompts for accelerated inference of large language models," in *Proc. EMNLP*, 2023, arXiv:2310.05736.

[2] H. Jiang, Q. Wu, X. Luo, D. Li, C.-Y. Lin, Y. Yang, and L. Qiu, "LongLLMLingua: Accelerating and enhancing LLMs in long context scenarios via prompt compression," in *Proc. ACL*, 2024, arXiv:2310.06839.

[3] Z. Pan *et al.*, "LLMLingua-2: Data distillation for efficient and faithful task-agnostic prompt compression," in *Findings of ACL*, 2024, arXiv:2403.12968.

[4] M. Yuksekgonul, F. Bianchi, J. Boen, S. Liu, Z. Huang, C. Guestrin, and J. Zou, "TextGrad: Automatic 'differentiation' via text," arXiv:2406.07496, 2024.

[5] Y. Li, B. Dong, F. Guerin, and C. Lin, "Compressing context to enhance inference efficiency of large language models," in *Proc. EMNLP*, 2023, arXiv:2310.06201.

[6] L. Zheng *et al.*, "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in *Proc. NeurIPS Datasets and Benchmarks*, 2023, arXiv:2306.05685.

[7] Z. Yang *et al.*, "HotpotQA: A dataset for diverse, explainable multi-hop question answering," in *Proc. EMNLP*, 2018.

[8] OpenAI, "gpt-oss-120b and gpt-oss-20b model card," arXiv:2508.10925, 2025.

[9] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Trans. Assoc. Comput. Linguistics*, vol. 12, 2024, arXiv:2307.03172.

[10] A. Madaan *et al.*, "Self-Refine: Iterative refinement with self-feedback," in *Proc. NeurIPS*, 2023, arXiv:2303.17651.

[11] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Proc. NeurIPS*, 2023, arXiv:2303.11366.